

Admin_Views

View 1 — `view_set_count`

Explanation (line-by-line)

- `CREATE OR REPLACE VIEW view_set_count AS` — Creates a named view. If it already exists it replaces it. A view is a saved `SELECT` query presented as a virtual table.
 - `SELECT` — Starts the list of columns the view will return.
 - `up.name as user_name` — `up` is an alias for `user_profile`; this picks the name column and renames it to `user_name` for readability in output.
 - `COUNT(tr.record_id) as total_workouts` — `COUNT()` is an aggregation that counts the number of rows where `tr.record_id` is not `NULL`; this gives total workout records per user.
 - `COUNT(DISTINCT tr.exercise_id) as unique_exercises` — Counts distinct exercise IDs to find how many different exercises the user performed; `DISTINCT` avoids double counting the same exercise.
 - `FROM user_profile up` — The main table for this query is `user_profile`, and `up` is a short alias so columns can be referenced as `up.col`.
 - `LEFT JOIN training_records tr ON up.user_id = tr.user_id` — Joins users to their training records. `LEFT JOIN` ensures users without records are included with `NULL`s on the `tr` side (important when aggregating).
 - `GROUP BY up.user_id, up.name` — Tells SQL to aggregate rows by user; aggregates (`COUNT`) are computed per group. Group by `user_id` (unique) and name (friendly label).
 - `HAVING COUNT(tr.record_id) > 0` — Filters groups after aggregation: only keep users who have more than 0 training records. (Note: `WHERE` can't filter on aggregated counts, so `HAVING` is used.)
 - `ORDER BY total_workouts DESC` — Sorts results so users with more workouts appear first.
-

View 2 — `view_set_sum`

Explanation (line-by-line)

- `CREATE OR REPLACE VIEW view_set_sum AS` — Creates or replaces the view named `view_set_sum`.
 - `up.name as user_name` — Same aliasing as before; readable column label.
 - `SUM(tr.sets) as total_sets` — `SUM()` adds up the sets values for each group (user). If a user has no rows, `SUM` would return `NULL` unless handled.
 - `SUM(tr.reps) as total_reps` — Adds up all reps for the user.
 - `SUM(tr.duration_minutes) as total_duration_minutes` — Sums total duration in minutes.
 - `FROM user_profile up` — Starting table.
 - `JOIN training_records tr ON up.user_id = tr.user_id` — `JOIN` (same as `INNER JOIN`) requires matching rows in both tables; users without records won't appear in this view.
 - `GROUP BY up.user_id, up.name` — Aggregate per user.
 - `ORDER BY total_sets DESC` — Show users who performed the most sets first.
-

View 3 — `view_set_avg`

Explanation (line-by-line)

- `CREATE OR REPLACE VIEW view_set_avg AS` — Creates the `view_set_avg` view.
- `e.exercise_name, e.muscle_group` — Selects columns from exercises table (alias `e`). These identify each exercise.
- `ROUND(AVG(tr.sets), 2) as avg_sets` — `AVG()` computes average sets per exercise; `ROUND(..., 2)` rounds the result to 2 decimal places for neat display.
- `ROUND(AVG(tr.reps), 2) as avg_reps` — Average reps per exercise.
- `ROUND(AVG(tr.weight_used), 2) as avg_weight` — Average weight used for that exercise.
- `ROUND(AVG(tr.duration_minutes), 2) as avg_duration` — Average duration.
- `FROM exercises e` — The main table is exercises, aliased as `e`.

- `JOIN training_records tr ON e.exercise_id = tr.exercise_id` — Inner join ensures only exercises that have training records are included.
- `GROUP BY e.exercise_id, e.exercise_name, e.muscle_group` — Group by exercise (ID is the unique key; name and muscle group are included so they can appear in SELECT).
- `HAVING COUNT(tr.record_id) >= 1` — Ensures the exercise has at least one training record. (This is redundant given the INNER JOIN but makes intention clear.)
- `ORDER BY avg_weight DESC` — Sort exercises by average weight used, highest first.

View 4 — `view_set_max_min`

Explanation (line-by-line)

- `CREATE OR REPLACE VIEW view_set_max_min AS` — Creates the view for min/max stats.
- `e.exercise_name` — Label for the exercise.
- `MIN(tr.weight_used) as min_weight` — `MIN()` returns the smallest non-NULL weight used for that exercise.
- `MAX(tr.weight_used) as max_weight` — `MAX()` returns the largest weight used.
- `MIN(tr.sets) as min_sets / MAX(tr.sets) as max_sets` — Minimum and maximum sets recorded for that exercise.
- `MIN(tr.reps) as min_reps / MAX(tr.reps) as max_reps` — Min and max reps.
- `FROM exercises e` — Exercises table as e.
- `JOIN training_records tr ON e.exercise_id = tr.exercise_id` — Inner join to include only exercises with training rows.
- `WHERE tr.weight_used IS NOT NULL` — Filters out records where `weight_used` is NULL so MIN/MAX of weight considers only meaningful values.
- `GROUP BY e.exercise_id, e.exercise_name` — Grouping by exercise.
- `ORDER BY max_weight DESC` — Shows exercises with highest maximum weight first.

View 5 — `view_set_muscle_group_stats`

Explanation (line-by-line)

- `CREATE OR REPLACE VIEW view_set_muscle_group_stats AS` — Creates a view that aggregates by muscle group.
 - `e.muscle_group` — The column we group by; could be values like 'Chest', 'Back', 'Legs'.
 - `COUNT(DISTINCT e.exercise_id) as exercise_count` — Counts how many distinct exercises belong to each muscle group.
 - `COUNT(tr.record_id) as total_workouts` — Total number of training records linked to exercises in that muscle group.
 - `ROUND(AVG(tr.sets), 2) as avg_sets` — Average sets across workouts for that muscle group, rounded.
 - `ROUND(AVG(tr.reps), 2) as avg_reps` — Average reps.
 - `SUM(tr.duration_minutes) as total_minutes` — Total duration across all workouts for that muscle group.
 - `FROM exercises e` — Exercises table is left side.
 - `LEFT JOIN training_records tr ON e.exercise_id = tr.exercise_id` — Left join makes sure muscle groups with exercises but no workouts still appear (though `HAVING` later will filter out unused groups).
 - `GROUP BY e.muscle_group` — Aggregate by the muscle group value.
 - `HAVING COUNT(tr.record_id) > 0` — Keep only groups that have at least one workout; removes muscle groups with zero activity.
 - `ORDER BY total_workouts DESC` — Order muscle groups by activity level.
-

Extra short points to say in viva (one-liners)

- **View:** “A saved SELECT query that behaves like a virtual table.”
- **Alias (e, up, tr):** “Short names for tables so queries are shorter and clear when joining multiple tables.”
- **LEFT JOIN vs JOIN (INNER JOIN):** “LEFT JOIN keeps all rows from left table; INNER JOIN requires matches on both sides.”
- **GROUP BY:** “Groups rows so aggregate functions compute per group.”

- **HAVING:** “Filters groups after aggregation (WHERE filters rows before grouping).”
 - **COUNT/SUM/AVG/MIN/MAX:** “Basic aggregation functions for counts, sums, averages, minima, and maxima.”
 - **ROUND(x,2):** “Rounds numeric results to 2 decimal places for neat presentation.”
-